

mongoDB®

MANUAL

Xavier Pérez

Índex

1. Introducció a MongoDB.....	3
1.1. Definició de BBDD NoSQL.....	3
1.2. Tipus de dades NoSQL.....	3
1.3. Avantatges i desavantatges en front a BBDD de dades relacionals.....	4
1.4. Explicar que es MongoDB i per què es útil.....	5
2. Creació de l'estructura de dades a MongoDB.....	7
2.1. Com és l'estructura de dades a MongoDB.....	7
2.2. Iniciar MongoDB mitjançant Docker.....	8
2.3. Creació de coleccions.....	10
2.4. Com crear documents.....	10
2.5. Crear validaments.....	11
3. Operacions CRUD.....	12
3.1. Inserir documents.....	12
3.2. Consulta bàsica.....	13
3.3. Cerca per criteris simples.....	14
3.4. Ordenar resultats.....	15
3.5. Operacions de consulta.....	17
a) Comparació.....	17
b) Operadors lògics.....	18
c) Operadors d'element a un array.....	18
3.6. Consultes amb expressions regulars.....	19
3.7. Actualització de documents d'un valor.....	19
3.8. Actualització de documents amb molts valors.....	19
3.9. Eliminar dades document d'un valor.....	20
3.10. Eliminar dades document amb molts valors.....	20
3.11. Comparació amb altres BBDD no-SQL.....	21

1. Introducció a MongoDB

1.1. Definició de BBDD NoSQL

Les BBDD NoSQL (not only SQL) es caracteritzen emmagatzemar informació en col·leccions en comptes de en taules i no utilitzar el llenguatge SQL, d'aquesta manera es poden emmagatzemar dades de forma més versàtil i lleugera al no tenir relacions i per tant poder accedir a les dades de forma més fàcil.

1.2. Tipus de dades NoSQL

1. Document: Aquest tipus emmagatzema dades en documents, generalment en formats com JSON o BSON. És flexible i permet emmagatzemar diverses estructures de dades en cada document.

2. Columnar: Aquest tipus emmagatzema les dades en columnes, en lloc de files. És ideal per a consultes analítiques i dades amb esquemes variables.

3. Key-Value: Aquest tipus emmagatzema dades com a parells clau-valor. És simple i eficient per a operacions bàsiques de lectura i escriptura.

4. Grafs: Aquest tipus emmagatzema dades com a nodes i relacions entre ells. És ideal per a representar interconnexions complexes entre les dades.

5. Objecte: Aquest tipus emmagatzema objectes complexos, com ara objectes Java o .NET, sense necessitat de mapatge a taules.

6. Column-Family: Aquest tipus organitza les dades en files i columnes, similar a les bases de dades de columnes, però agrupa les columnes en famílies que poden ser accedides conjuntament.

1.3. Avantatges i desavantatges en front a BBDD de dades relacionals

Avantatges de les bases de dades NoSQL:

1. **Escalabilitat horitzontal:** Poden gestionar grans volums de dades distribuïdes eficientment.
2. **Flexibilitat de l'esquema:** Són adaptables als canvis en els requeriments de les dades.
3. **Rendiment:** Ofereixen un bon rendiment per a operacions específiques.

Desavantatges de les bases de dades NoSQL:

1. **Consistència eventual:** Hi pot haver un petit retard en la propagació d'actualitzacions.
2. **Manca de suport transaccional:** Moltes no ofereixen suport transaccional ACID.
3. **Manca d'eines madures:** Les eines de gestió poden ser menys desenvolupades comparades amb les bases de dades relacionals.

1.4. Explicar que es MongoDB i per què es útil

MongoDB es un sistema de BBDD orientat a documents que desa estructures de dades BSON amb un esquema dinàmic, fent la integració de les dades fàcil i ràpida.

Característiques principals:

- **Flexibilitat del Schema:** Permet emmagatzemar dades en documents JSON flexibles, el que significa que les columnes poden tenir diferents camps i no s'han de seguir estrictament un esquema predefinit.
- **Alta Disponibilitat:** Suporta rèpliques, permetent la replicació de dades en diferents nodes, el que assegura la disponibilitat de les dades fins i tot en cas de fallada d'un servidor.
- **Escalabilitat Horitzontal:** Es pot escalar fàcilment afegint més nodes al clúster, el que millora el rendiment i la capacitat de gestió de càrregues de treball pesades.
- **Indexació Potent:** Permet crear índexos sobre qualsevol camp en una col·lecció, el que millora la velocitat de les consultes i la capacitat de recuperació de dades.
- **Consulta Rica:** Suporta consultes complexes mitjançant l'ús de l'API de consultes, incloent-hi consultes ad hoc, consultes de rang i agregacions.
- **Càrregues de Treball Distribuïdes:** Proporciona suport per a càrregues de treball distribuïdes mitjançant la fragmentació de dades a través de múltiples nodes, el que permet gestionar grans volums de dades.
- **Transaccions:** Suporta transaccions ACID a través de múltiples documents i col·leccions, el que assegura la consistència de les dades en entorns d'alta disponibilitat.
- **Alt Rendiment:** Optimitzat per a treballs de lectura i escriptura ràpids, amb suport per a operacions en memòria cau per millorar el rendiment.

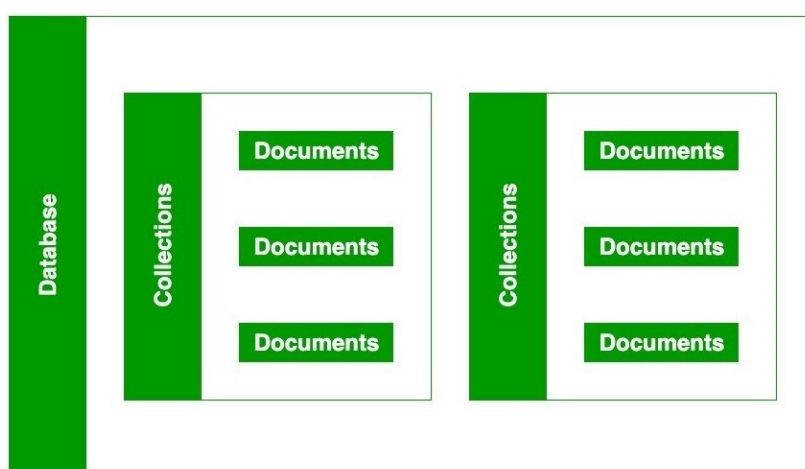
- **Escalabilitat Global:** MongoDB Atlas ofereix una opció de desplegament en núvol que permet escalar bases de dades a nivell global amb facilitat, assegurant un accés ràpid i fiable des de qualsevol ubicació.

2. Creació de l'estructura de dades a MongoDB

2.1. Com és l'estructura de dades a MongoDB

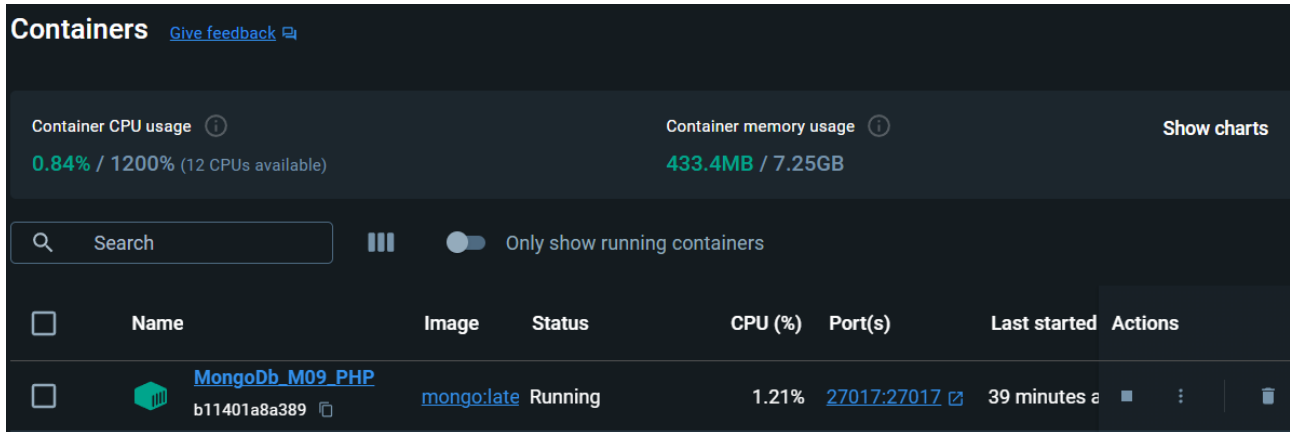
MongoDB consta de 4 diferents nivells

- **Base de dades:** El repositori on es guarden els arxius.
- **Coleccions:** Els grups en els que es divideixen les dades, el que seria una taula.
- **Documents:** Els grups d'informació dins de les coleccions.
- **Camps:** Dins dels documents es la dada en si mateixa, el que pot ser un nom, una data, un telèfon etc...



2.2. Iniciar MongoDB mitjançant Docker

Primer tenim que tenir el container correctament iniciat a Docker.



Name	Image	Status	CPU (%)	Port(s)	Last started	Actions
MongoDb_M09_PHP b11401a8a389	mongo:late	Running	1.21%	27017:27017	39 minutes a	[Stop] [Refresh] [Delete]

També ho podem comprovar mitjançant el cmd amb la comanda:

> «docker ps»

```
PS C:\Users\Xavier> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
b11401a8a389   mongo:latest   "docker-entrypoint.s..." 6 days ago    Up 40 minutes  0.0.0.0:27017->27017/tcp          MongoDb_M09_PHP
PS C:\Users\Xavier>
```

Per connectar-nos al container executem la comanda:

> «docker exec -it <nom del container> bash»

I per començar a posar comandes a la BBDD posem la comanda:

> «mongosh»


```

C:\Users\Xavier>docker exec -it MongoDB_M09_PHP bash
root@b11401a8a389:/# mongosh
Current Mongosh Log ID: 6626852e97a1e9bf58c934dc
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.2.4
Using MongoDB:      7.0.8
Using Mongosh:      2.2.4

For mongosh info see: https://docs.mongodb.com/mongosh-shell/

-----
The server generated these startup warnings when booting
2024-04-22T15:04:09.780+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
2024-04-22T15:04:11.463+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2024-04-22T15:04:11.463+00:00: /sys/kernel/mm/transparent_hugepage/enabled is 'always'. We suggest setting it to 'never' in this binary version
2024-04-22T15:04:11.463+00:00: vm.max_map_count is too low
-----
test>

```

Per començar tindrem que crear una BBDD amb la comanda:

> «use <nom BBDD>»

(Aquesta comanda també serveix per seleccionar una BBDD existent)

```

test> use proves
switched to db proves

```

I per veure les nostres BBDD executem la comanda:

> «show dbs»

(No sortirà la BBDD creada fins anyadir dades)

```

proves> show dbs
admin      40.00 KiB
config     72.00 KiB
cuacker    80.00 KiB
local      72.00 KiB
proves>

```

2.3. Creació de col·leccions

Per crear una col·lecció fem servir la comanda:

> «db.createCollection("<nom col·lecció>")»

```
proves> db.createCollection("Coleccio1")
{ ok: 1 }
proves>
```

Per eliminar-la fem el següent:

> «db.<nom col·lecció>.drop()»

```
proves> db.Coleccio1.drop()
true
proves>
```

2.4. Com crear documents

Per crear un sol document a una col·lecció tenim la següent comanda:

> «db.<nom col·lecció>.insertOne({<camp 1>: <valor 1>, <camp 2>: <valor 2>, ...})»

```
proves> db.Coleccio1.insertOne({nom: "Xavier", edat: 19, ciutat: "Martorell"})
{
  acknowledged: true,
  insertedId: ObjectId('6627cdba5b3f31a295c934dd')
}
proves> |
```

També podem crear varis documents d'un cop:

> «db.<nom col·lecció>.insertMany([<camp 1>: <valor 1>, <camp 2>: <valor 2>, ...],
[<camp 1>: <valor 1>, <camp 2>: <valor 2>, ...]])»

```
proves> db.Coleccio1.insertMany([{"nombre": "Maria", edad: 19, ciudad: "Martorell"},
{"nombre": "Aaron", edad: 22, ciudad: "Martorell"}])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6627cf2b5b3f31a295c934de'),
    '1': ObjectId('6627cf2b5b3f31a295c934df')
  }
}
proves>
```

2.5. Crear validaments

Amb els validaments podem crear una seria de normes que han de seguir els documents dins d'una col·lecció, aquí es pot veure un exemple:

En aquest cas creo directament una col·lecció amb la seva validació:

```
proves> db.createCollection("exemple", { validator: { $jsonSchema: { bsonType: "object", required: ["nom"],  
properties: { nom: { bsonType: "string", description: "Ha de ser un String amb el nom" } } } } })  
{ ok: 1 }  
proves>
```

També es pot aplicar a una col·lecció ja existent canviant una mica la comanda, aquí dono un exemple de com seria:

```
db.runCommand({  
  collMod: "usuaris",  
  validator: {  
    $jsonSchema: {  
      bsonType: "object",  
      required: ["nom", "edat", "ciutat"],  
      properties: {  
        nom: {  
          bsonType: "string",  
          description: "ha de ser una cadena i és obligatori"  
        },  
        edat: {  
          bsonType: "int",  
          minimum: 0,  
          description: "ha de ser un enter positiu i és obligatori"  
        },  
        ciutat: {  
          bsonType: "string",  
          description: "ha de ser una cadena i és obligatori"  
        }  
      }  
    }  
  }  
})
```

3. Operacions CRUD

Les operacions CRUD són el conjunt bàsic de operacions que es fan servir en BBDD.

CRUD és l'acrònim de les quatre operacions fonamentals: Create, Read, Update i Delete.

3.1. Inserir documents

La inserció es bàsicament la creació de documents com ho hem vist abans.

Per crear un sol document a una col·lecció tenim la següent comanda:

> «db.<nom col·lecció>.insertOne({<camp 1>: <valor 1>, <camp 2>: <valor 2>, ...})»

```
proves> db.Coleccio1.insertOne({nom: "Xavier", edat: 19, ciutat: "Martorell"})
{
  acknowledged: true,
  insertedId: ObjectId('6627cdba5b3f31a295c934dd')
}
proves> |
```

També podem crear varis documents d'un cop:

> «db.<nom col·lecció>.insertMany([<camp 1>: <valor 1>, <camp 2>: <valor 2>, ...],
[<camp 1>: <valor 1>, <camp 2>: <valor 2>, ...]])»

```
proves> db.Coleccio1.insertMany([{"nombre": "Maria", edad: 19, ciudad: "Martorell"},
{"nombre": "Aaron", edad: 22, ciudad: "Martorell"}])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6627cf2b5b3f31a295c934de'),
    '1': ObjectId('6627cf2b5b3f31a295c934df')
  }
}
proves>
```

3.2. Consulta bàsica

Per veure tot el contingut d'una col·lecció fem el següent:

> «db.<nom col·lecció>.find()»

```
proves> db.Coleccio1.find()
[
  {
    _id: ObjectId('6627cdba5b3f31a295c934dd'),
    nom: 'Xavier',
    edat: 19,
    ciutat: 'Martorell'
  },
  {
    _id: ObjectId('6627cf2b5b3f31a295c934de'),
    nombre: 'Maria',
    edad: 19,
    ciudad: 'Martorell'
  },
  {
    _id: ObjectId('6627cf2b5b3f31a295c934df'),
    nombre: 'Aaron',
    edad: 22,
    ciudad: 'Martorell'
  }
]
proves>
```

3.3. Cerca per criteris simples

Per fer una cerca amb filtre fem el següent:

> «db.<nom col·lecció>.find({<camp>:<valor>})»

```
proves> db.Coleccio1.find({ edat: 19 })
[
  {
    _id: ObjectId('6627cdba5b3f31a295c934dd'),
    nom: 'Xavier',
    edat: 19,
    ciutat: 'Martorell'
  },
  {
    _id: ObjectId('6627d7ba5b3f31a295c934e0'),
    nom: 'Maria',
    edat: 19,
    ciutat: 'Martorell'
  }
]
proves>
```

3.4. Ordenar resultats

Podem ordenar de forma ascendent i descendent utilitzant aquesta comanda:

> «db.<nom col·lecció>.find().sort({ <camp> : <1 o -1> })»

(1 per ordre ascendent i -1 per orde descendent)

```
proves> db.Coleccio1.find().sort({edat: -1})
[
  {
    _id: ObjectId('6627d7ba5b3f31a295c934e1'),
    nom: 'Aaron',
    edat: 22,
    ciutat: 'Martorell'
  },
  {
    _id: ObjectId('6627cdba5b3f31a295c934dd'),
    nom: 'Xavier',
    edat: 19,
    ciutat: 'Martorell'
  },
  {
    _id: ObjectId('6627d7ba5b3f31a295c934e0'),
    nom: 'Maria',
    edat: 19,
    ciutat: 'Martorell'
  }
]
proves>
```

3.5. Operacions de consulta

a) Comparació

- \$gt: Major que.
- \$lt: Menor que.
- \$gte: Major o igual que.
- \$lte: menor o igual que.
- \$in: coincideix amb qualsevol valor a l'array especificat.
- \$nin: No coincideix amb cap valor a l'array especificat.
- \$or: coincideix amb qualsevol de les condicions especificades.
- \$and: coincideix amb totes les condicions especificades.

Un exemple seria el següent:

> «db.<nom col·lecció>.find({ <camp>: { \$gt: 30 }, <camp>: "<valor>" })»

```
proves> db.Coleccio1.find({ edat: { $gt: 20 }, ciutat: "Martorell" })
[
  {
    _id: ObjectId('6627d7ba5b3f31a295c934e1'),
    nom: 'Aaron',
    edat: 22,
    ciutat: 'Martorell'
  }
]
```


b) Operadors lògics

Els operadors lògics us permeten combinar múltiples condicions en una sola consulta. Alguns dels operadors lògics comuns són:

- `$and`: Combina múltiples condicions i torna documents que compleixen totes les condicions.
- `$or`: Combina múltiples condicions i torna documents que compleixen almenys una de les condicions.

Exemple:

```
> «db.<nom col·lecció>.find({ $and:[{ <camp>: { $gt: 30 } }, { <camp>: "<valor>" }] })»
```

c) Operadors d'element a un array

Aquests operadors s'utilitzen per fer consultes en camps que són arranjaments. Alguns dels operadors comuns són:

- `$in`: Seleccioneu els documents on el valor d'un camp es troba en una llista especificada.
- `$nin`: Seleccioneu els documents on el valor d'un camp no es troba en una llista especificada.

Exemple:

```
> «db.<nom col·lecció>.find({ <camp>: { $in: ["vermell", "blau"] } })»
```

3.6. Consultes amb expressions regulars

Pots cercar patrons en qualsevol posició de la cadena utilitzant expressions regulars.

Per exemple, si vols trobar tots els documents on el camp "email" contingui la paraula "gmail", ho pots fer així:

```
> «db.<nom col·lecció>.find({ email: { $regex: /gmail/ } })»
```

3.7. Actualització de documents d'un valor

Per actualitzar un sol document, podeu utilitzar el mètode `updateOne()`. Aquest mètode pren dos arguments: un filtre que especifica quin document actualitzar i una operació d'actualització que descriu com modificar el document.

Exemple:

```
> «db.<nom col·lecció>.updateOne(  
  { <camp>: "<valor>" }, // Filtro per saber els documents a actualitzar  
  { $set: { <camp>: <valor> } } // Actualització  
)»
```

3.8. Actualització de documents amb molts valors

Si voleu actualitzar múltiples documents amb múltiples valors, utilitzeu el mètode `updateMany()` amb un filtre per seleccionar els documents que voleu actualitzar i un objecte que contingui els camps i valors que voleu actualitzar utilitzant l'operador `$set`.

Exemple:

```
> «db.<nom col·lecció>.updateMany(  
  { <camp>: <valor> }, // Filtro per saber els documents a actualitzar  
  { $set: { <camp>: <valor>, <camp>: "<valor>" } } // Actualització  
)»
```

3.9. Eliminar dades document d'un valor

Per eliminar un document complet a MongoDB, podeu utilitzar el mètode `deleteOne()`. No obstant això, tingueu en compte que `deleteOne()` elimina el document sencer, no només un camp específic dins del document.

Exemple:

```
> «db.<nom col·lecció>.deleteOne(  
  { <camp>: "<valor>" } // Filtro per saber els documents a eliminar.  
)»
```

3.10. Eliminar dades document amb molts valors

Si vols eliminar múltiples documents que compleixin certs criteris, pots fer servir el mètode `deleteMany()` de manera similar.

Exemple:

```
> «db.<nom col·lecció>.deleteMany(  
  { <camp>: "<valor>" }, // Filtro per saber els documents a eliminar.  
  { $unset: { <camp>: "", <valor>: "" } } // Elimina els camps dels documents seleccionats.  
)»
```

3.11. Comparació amb altres BBDD no-SQL

Comparació amb altres bases de dades NoSQL com Couchbase i Cassandra:

1. Model de Dades:

- MongoDB utilitza un model de dades basat en documents, on les dades s'emmagatzemen en documents JSON flexibles.
- Couchbase també utilitza un model de dades basat en documents amb el format JSON.
- Cassandra utilitza un model de dades basat en columnes, que és adequat per a aplicacions amb necessitats d'escriptura intensiva.

2. Escalabilitat:

- MongoDB ofereix escalabilitat horitzontal mitjançant la fragmentació de dades i la replicació.
- Couchbase també ofereix escalabilitat horitzontal amb l'ús de nodes i clústers.
- Cassandra és coneguda per la seva alta escalabilitat lineal, especialment per a grans volums de dades.

3. Consistència i Disponibilitat:

- MongoDB ofereix una consistència eventual per defecte, però es pot configurar per proporcionar consistència fortament eventual.
- Couchbase ofereix una consistència consistent mitjançant la replicació sincronitzada.
- Cassandra permet als desenvolupadors triar entre diferents nivells de consistència, des de molt coherents fins a eventualment coherents.

4. Consulta i Índex:

- MongoDB suporta consultes ad hoc mitjançant un llenguatge de consulta ric i pot indexar camps específics dels documents.
- Couchbase utilitza N1QL (un llenguatge de consulta basat en SQL) per a consultes.
- Cassandra requereix que es dissenyin les taules per respondre a consultes específiques, i ofereix índexos secundaris per millorar el rendiment.

5. Escenaris d'ús:

- MongoDB és adequat per a aplicacions que necessiten flexibilitat en l'estructura de les dades i un desenvolupament ràpid.
- Couchbase és útil per a aplicacions que requereixen un rendiment escalable i un accés ràpid a les dades amb l'ús de memòria cau.
- Cassandra és ideal per a aplicacions amb moltes escriptures i lectures simultànies distribuïdes.

En resum, mentre que MongoDB, Couchbase i Cassandra són totes bases de dades NoSQL, cadascuna té els seus avantatges i és adequada per a diferents casos d'ús i requisits de l'aplicació.